

COMPUTER NETWORKS COURSE PROJECT

Private Network Service Platform — Phase 1

Team Members

- **Vani Rudra** — Mac 1: Private DNS + Backend A + Client
- **Apoorva Choudhary** — Mac 2: Nginx + HTTPS + Load Balancer + Client
- **Vidhi** — Mac 3: Backend B + Client

1. Project Overview

The objective of this project is to build a completely local private network service platform using three macOS laptops.

The system demonstrates the complete flow:

DNS → TCP → TLS → HTTP → Backend

The client accesses the application using the private domain:

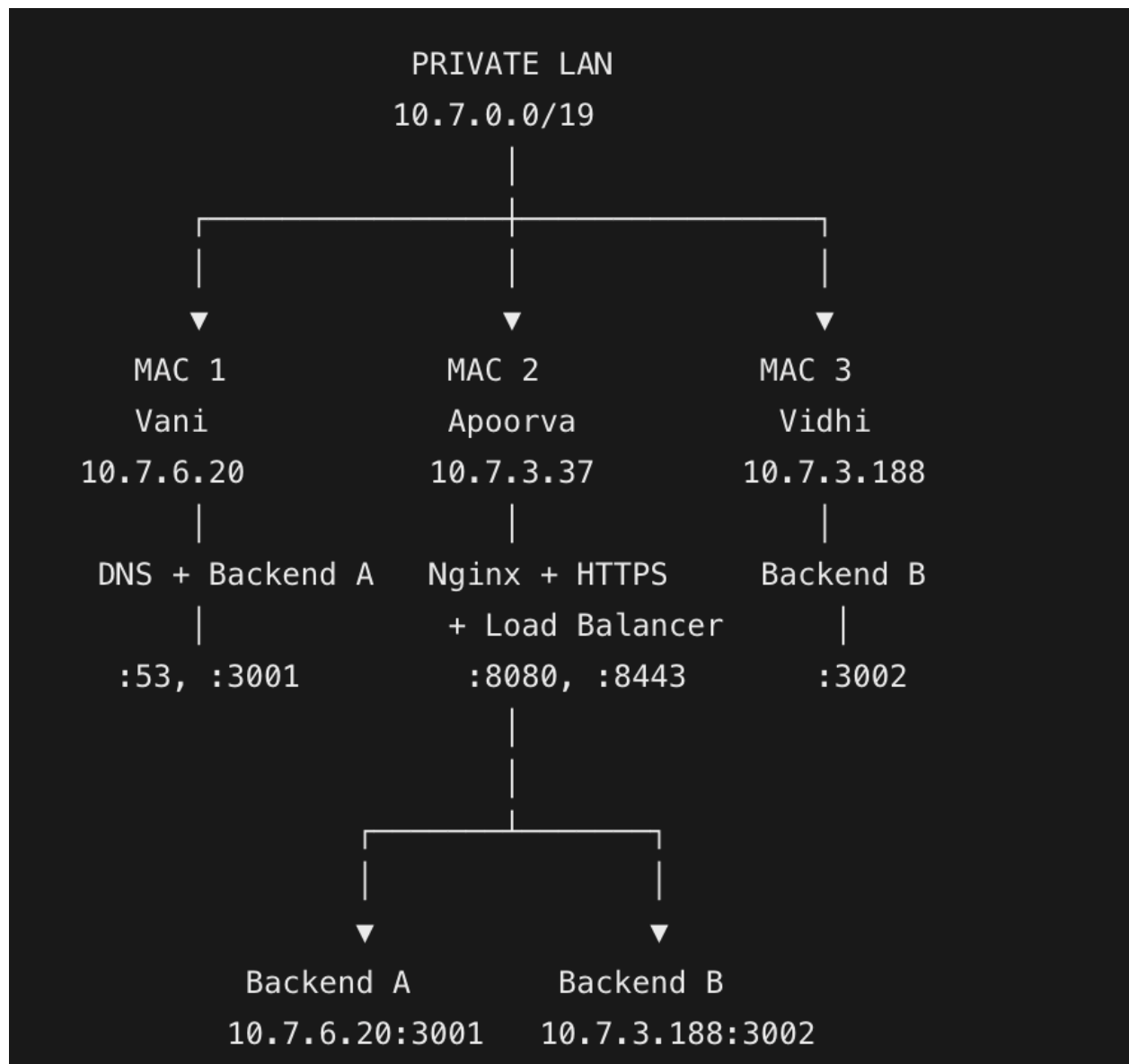
[app.teambyte.test](#)

The request is first resolved by our private DNS server, then sent to the Nginx reverse proxy/load balancer, which forwards the request to either Backend A or Backend B.

The project uses **.test** as the private domain namespace, as required by the project specification.

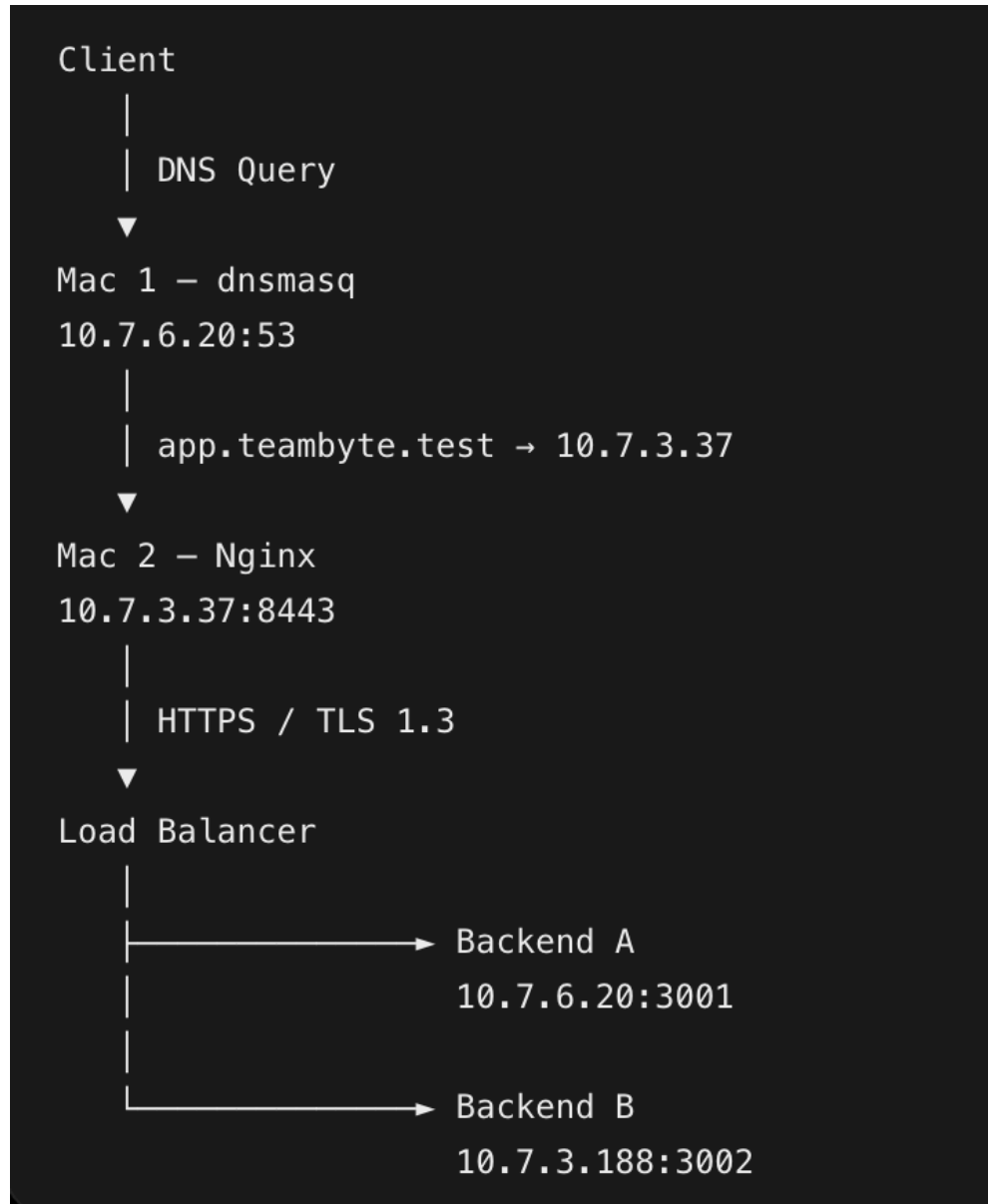
2. Team Architecture

Our three-machine architecture is:



Request flow

The actual project flow is:



3. Machine and Service Inventory

Machine	IP Address	Interface	MAC Address	Role	Services
Mac 1 — Vani	10.7.6.20	en0	10:9f:41:b7:1b:c8	DNS + Backend A + Client	DNS 53, Backend A 3001
Mac 2 — Apoorva	10.7.3.37	en0	10:9f:41:c5:c4:5c	Nginx + HTTPS + Load Balancer + Client	HTTP 8080, HTTPS 8443
Mac 3 — Vidhi	10.7.3.188	en0	f8:73:df:f4:00:56	Backend B + Client	Backend B 3002

Network details:

- Subnet Mask: 255.255.224.0
- Prefix: /19
- Default Gateway: 10.7.0.1
- Network: 10.7.0.0/19

4. Task A — Establish the Private LAN

All three Macs are connected to the same private Wi-Fi/LAN network.

The project requires recording the private IP address, subnet, gateway, active interface and MAC address, along with verifying reachability between machines using `ping`.

4.1 Mac 1 Network Configuration



COMMAND

```
networksetup -getinfo "Wi-Fi"
```



OUTPUT

```
DHCP Configuration
IP address: 10.7.6.20
Subnet mask: 255.255.224.0
Router: 10.7.0.1
IPv6: Automatic
IPv6 IP address: none
IPv6 Router: none
Wi-Fi ID: 10:9f:41:b7:1b:c8
```

Active Interface



COMMAND `route -n get default | grep interface`



OUTPUT

```
interface: en0
```

Mac-1 (Vani)

```
[DHCP Configuration
IP address: 10.7.6.20
Subnet mask: 255.255.224.0
Router: 10.7.0.1
Client ID:
IPv6: Automatic
IPv6 IP address: none
IPv6 Router: none
Wi-Fi ID: 10:9f:41:b7:1b:c8
vanirudra@Vanis-MacBook-Pro-3 ~ % route -n get default | grep interface
interface: en0
```

Mac-3 (Vidhi)

```
~ % sudo ifconfig en0 status . OK
DHCP Configuration
IP address: 10.7.3.188
Subnet mask: 255.255.224.0
Router: 10.7.0.1
Client ID:
IPv6: Automatic
IPv6 IP address: none
IPv6 Router: none
Wi-Fi ID: f8:73:df:f4:00:56
vidhi@vidhis-MacBook-Pro ~ % route -n get default | grep interface
interface: en0
```

5. LAN Connectivity Testing

7.1 Vani → Apoorva



COMMAND

```
ping -c 4 10.7.3.37
```



OUTPUT

```
4 packets transmitted, 4 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 19.662/54.910/101.009/31.864 ms
```

Result: Successful communication with 0% packet loss.

```
[vanirudra@Vanis-MacBook-Pro-3 ~ % ping -c 4 10.7.3.37  
PING 10.7.3.37 (10.7.3.37): 56 data bytes  
64 bytes from 10.7.3.37: icmp_seq=0 ttl=64 time=9.790 ms  
64 bytes from 10.7.3.37: icmp_seq=1 ttl=64 time=119.821 ms  
64 bytes from 10.7.3.37: icmp_seq=2 ttl=64 time=6.869 ms  
64 bytes from 10.7.3.37: icmp_seq=3 ttl=64 time=66.911 ms  
  
--- 10.7.3.37 ping statistics ---  
4 packets transmitted, 4 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 6.869/50.848/119.821/46.463 ms  
vanirudra@Vanis-MacBook-Pro-3 ~ % █
```

7.2 Vani → Vidhi

Command

```
>_ COMMAND ping -c 4 10.7.3.188
```

```
>_ OUTPUT
```

```
4 packets transmitted, 4 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 70.535/128.776/177.354/38.074 ms
```

Result: Successful communication with 0% packet loss.

```
[vanirudra@Vanis-MacBook-Pro-3 ~ % ping -c 4 10.7.3.188  
PING 10.7.3.188 (10.7.3.188): 56 data bytes  
64 bytes from 10.7.3.188: icmp_seq=0 ttl=64 time=13.073 ms  
64 bytes from 10.7.3.188: icmp_seq=1 ttl=64 time=7.931 ms  
64 bytes from 10.7.3.188: icmp_seq=2 ttl=64 time=11.837 ms  
64 bytes from 10.7.3.188: icmp_seq=3 ttl=64 time=11.638 ms  
  
--- 10.7.3.188 ping statistics ---  
4 packets transmitted, 4 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 7.931/11.120/13.073/1.921 ms  
vanirudra@Vanis-MacBook-Pro-3 ~ %
```

7.3 Apoorva → Vidhi

```
>_ COMMAND
```

```
ping -c 4 10.7.3.188
```

```
>_ OUTPUT
```

```
4 packets transmitted, 4 packets received, 0.0% packet loss  
round-trip min/avg/max/stddev = 18.485/87.439/152.177/47.728 ms
```

Result: Successful communication with 0% packet loss.

```
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % ping -c 4 10.7.3.188
PING 10.7.3.188 (10.7.3.188): 56 data bytes
64 bytes from 10.7.3.188: icmp_seq=0 ttl=64 time=94.772 ms
64 bytes from 10.7.3.188: icmp_seq=1 ttl=64 time=8.879 ms
64 bytes from 10.7.3.188: icmp_seq=2 ttl=64 time=28.162 ms
64 bytes from 10.7.3.188: icmp_seq=3 ttl=64 time=8.223 ms

--- 10.7.3.188 ping statistics ---
4 packets transmitted, 4 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 8.223/35.009/94.772/35.422 ms
apoorvachoudhary@Apoorvas-MacBook-Pro ~ %
```

Task A Result

Task A — Private LAN: COMPLETE 

8. Task B — Configure Private DNS Server

Mac 1 runs dnsmasq as the team's private DNS server.

DNS records configured:

[app.teambyte.test](#) → [10.7.3.37](#)

[api.teambyte.test](#) → [10.7.3.37](#)

Mac 2 and Mac 3 are configured to use:

[10.7.6.20](#)

as their DNS resolver.

The project requires at least two other Macs to use Mac 1 as their DNS resolver.

9. Mac 2 DNS Configuration



COMMAND

```
networksetup -getdnsservers "Wi-Fi"
```



OUTPUT

```
10.7.6.20
```

```
Found: trip min/avg/max/stddev = 0.1220/00.007774/772/00.422 ms  
[apoorvachoudhary@Apoorvas-MacBook-Pro ~ % networksetup -getdnsservers "Wi-Fi"  
10.7.6.20  
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % █
```

10. Mac 3 DNS Configuration



COMMAND

```
networksetup -getdnsservers "Wi-Fi"
```



OUTPUT

```
10.7.6.20
```

```
vidhi@vidhis-MacBook-Pro ~ % networksetup -getdnsservers "Wi-Fi"  
10.7.6.20  
vidhi@vidhis-MacBook-Pro ~ % █
```

11. DNS Resolution Test

On Mac 2:



COMMAND

```
dig app.teambyte.test
```



OUTPUT

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR  
;; ANSWER SECTION:  
app.teambyte.test. 0 IN A 10.7.3.37  
;; SERVER: 10.7.6.20#53(10.7.6.20)
```

Explanation

The client asks the private DNS server at **10.7.6.20**.

The DNS server resolves:

app.teambyte.test



10.7.3.37

```
10.7.6.20
[apoorvachoudhary@Apoorvas-MacBook-Pro ~ % dig app.teambyte.test

; <<>> DiG 9.10.6 <<>> app.teambyte.test
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 47364
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.teambyte.test.                IN      A

;; ANSWER SECTION:
app.teambyte.test.                0       IN      A      10.7.3.37

;; Query time: 12 msec
;; SERVER: 10.7.6.20#53(10.7.6.20)
;; WHEN: Sat Oct 03 21:04:05 IST 2026
;; MSG SIZE rcvd: 62

apoorvachoudhary@Apoorvas-MacBook-Pro ~ %
```

Task B Result

Task B — Private DNS: COMPLETE 

12. Task C — Two Backend Services

The project requires two simple REST backends:

- Backend A → port 3001
- Backend B → port 3002

Both provide:

GET /

GET /api/status

and return an **X-Backend** header identifying the server.

13. Backend A

Machine: Mac 1

IP: 10.7.6.20

Port: 3001



COMMAND

```
curl -i http://10.7.6.20:3001/api/status
```



OUTPUT

```
HTTP/1.1 200 OK
X-Powered-By: Express
X-Backend: A
Cache-Control: max-age=60
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
{"backend":"A","status":"ok"}
```

`{"backend":"A","status":"ok"}`

```
[vanirudra@Vanis-MacBook-Pro-3 ~ % curl -i http://10.7.6.20:3001/api/status
HTTP/1.1 200 OK
X-Powered-By: Express
X-Backend: A
Cache-Control: max-age=60
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
Date: Sat, 03 Oct 2026 15:37:23 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"backend":"A","status":"ok"}%
vanirudra@Vanis-MacBook-Pro-3 ~ %
```

14. Backend B

Machine: Mac 3

IP: 10.7.3.188

Port: 3002



COMMAND

```
curl -i http://10.7.3.188:3002/api/status
```



OUTPUT

```
HTTP/1.1 200 OK
X-Powered-By: Express
X-Backend: B
Cache-Control: max-age=60
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-eqo46f6cZTMyh8pAIw5ntQG78PA"
{"backend":"B","status":"ok"}
```



OUTPUT · CONTINUED

```
ETag: W/"1d-eqo46f6cZTMyh8pAIw5ntQG78PA"  
{"backend":"B","status":"ok"}
```

10.7.6.20

```
[vidhi@vidhis-MacBook-Pro ~ % curl -i http://10.7.3.188:3002/api/status
```

```
HTTP/1.1 200 OK
```

```
X-Powered-By: Express
```

```
X-Backend: B
```

```
Cache-Control: max-age=60
```

```
Content-Type: application/json; charset=utf-8
```

```
Content-Length: 29
```

```
ETag: W/"1d-eqo46f6cZTMyh8pAIw5ntQG78PA"
```

```
Date: Sat, 03 Oct 2026 15:38:45 GMT
```

```
Connection: keep-alive
```

```
Keep-Alive: timeout=5
```

```
{  
  "backend": "B",  
  "status": "ok"  
}%
```

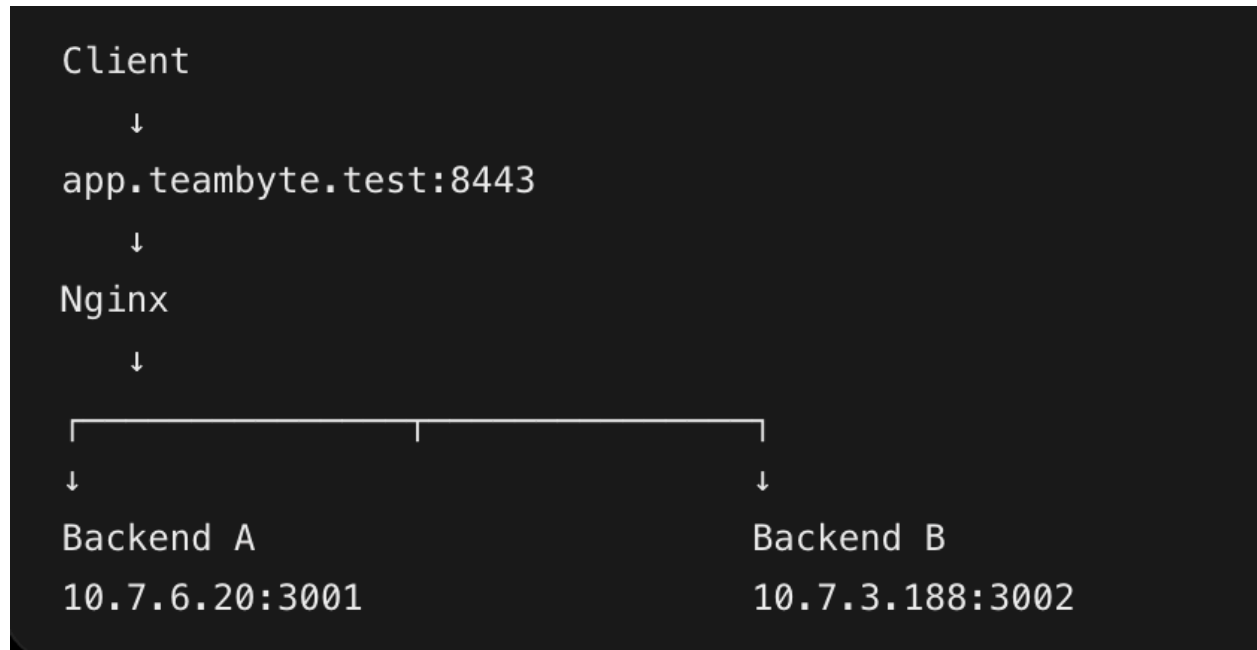
Task C Result

Task C — Backend Services: COMPLETE 

15. Task D — Reverse Proxy and Load Balancer

Mac 2 runs Nginx and acts as the single public entry point.

The architecture is:



Nginx uses round-robin load balancing.

The project requires repeated requests to show responses from both backends through the **X-Backend** header.

16. Load Balancing Test



COMMAND

```
for i in {1..10}; do
echo -n "Request $i: "
curl -s -D - https://app.teambyte.test:8443/api/status -o /dev/null |
grep -i "^X-Backend:"
done
```



OUTPUT

```
Request 1: X-Backend: A
Request 2: X-Backend: B
Request 3: X-Backend: A
Request 4: X-Backend: B
Request 5: X-Backend: A
Request 6: X-Backend: B
Request 7: X-Backend: A
Request 8: X-Backend: B
Request 9: X-Backend: A
Request 10: X-Backend: B
```

Explanation

The requests are distributed alternately between Backend A and Backend B.

This demonstrates round-robin load balancing.

```
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % for i in {1..10}; do
  echo -n "Request $i: "
  curl -s -D - https://app.teambyte.test:8443/api/status -o /dev/null | grep -i "^X-Backend:"
done
Request 1: X-Backend: A
Request 2: X-Backend: B
Request 3: X-Backend: A
Request 4: X-Backend: B
Request 5: X-Backend: A
Request 6: X-Backend: B
Request 7: X-Backend: A
Request 8: X-Backend: B
Request 9: X-Backend: A
Request 10: X-Backend: B
apoorvachoudhary@Apoorvas-MacBook-Pro ~ %
```

Task D Result

Task D — Reverse Proxy + Load Balancing: COMPLETE 

17. Task E — HTTPS / TLS

Nginx terminates TLS on:

HTTPS → port 8443

A self-signed certificate was generated for:

app.teambyte.test

api.teambyte.test

The certificate was added to the trust store of the client Macs.

The final HTTPS test is performed **without** **-k**, as required by the project.

18. HTTPS Certificate Verification



COMMAND

```
curl -v https://app.teambyte.test:8443/api/status
```



IMPORTANT OUTPUT

```
Connected to app.teambyte.test (10.7.3.37) port 8443
TLS handshake, Client hello
TLS handshake, Server hello
TLS handshake, Certificate
TLS handshake, CERT verify
TLS handshake, Finished
TLS handshake, Finished
```



OUTPUT · CONTINUED

```
SSL connection using TLSv1.3
subjectAltName: host "app.teambyte.test" matched cert's "app.teambyte.test"
SSL certificate verify ok.
HTTP/1.1 200 OK
Server: nginx/1.31.6
X-Backend: A
Cache-Control: max-age=60
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
{"backend": "A", "status": "ok"}
```

Explanation

The TLS handshake establishes a secure encrypted connection between the client and Nginx.

The certificate hostname matches `app.teambyte.test`, and the client trusts the certificate.

```

Request ID: X-Backend: B
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % curl -v https://app.teambyte.test:8443/api/status

* Host app.teambyte.test:8443 was resolved.
* IPv6: (none)
* IPv4: 10.7.3.37
* Trying 10.7.3.37:8443...
* Connected to app.teambyte.test (10.7.3.37) port 8443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted http/1.1
* Server certificate:
* subject: C=IN; ST=Haryana; L=Sonipat; O=TeamByte; OU=CN; CN=app.teambyte.test
* start date: Sep 29 17:58:43 2026 GMT
* expire date: Sep 29 17:58:43 2027 GMT
* subjectAltName: host "app.teambyte.test" matched cert's "app.teambyte.test"
* issuer: C=IN; ST=Haryana; L=Sonipat; O=TeamByte; OU=CN; CN=app.teambyte.test
* SSL certificate verify ok.
* using HTTP/1.x
> GET /api/status HTTP/1.1
> Host: app.teambyte.test:8443
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.6
< Date: Sat, 03 Oct 2026 15:43:28 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 29
< Connection: keep-alive
< X-Powered-By: Express
< X-Backend: A
< Cache-Control: max-age=60
< ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
<
* Connection #0 to host app.teambyte.test left intact
{"backend":"A","status":"ok"}%
apoorvachoudhary@Apoorvas-MacBook-Pro ~ %

```

Task E Result

Task E — HTTPS/TLS: COMPLETE 

19. Task F — HTTP Caching

The `/api/status` endpoint returns:

Cache-Control: max-age=60

and an ETag.

Initial response:

HTTP/1.1 200 OK

X-Backend: A

Cache-Control: max-age=60

ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"

20. Conditional Request



COMMAND

```
curl -i \  
-H 'If-None-Match: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"' \  
http://10.7.6.20:3001/api/status
```



OUTPUT

```
HTTP/1.1 304 Not Modified  
X-Powered-By: Express  
X-Backend: A  
Cache-Control: max-age=60  
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
```

ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"

There is no response body because the resource has not changed.

Explanation

The first request returned the resource and an ETag.

The client then sent the ETag using:

If-None-Match

Since the resource was unchanged, the server responded with:

304 Not Modified

This tells the client that it can reuse its cached copy.

The project specifically accepts a conditional **304 Not Modified** as the caching demonstration.

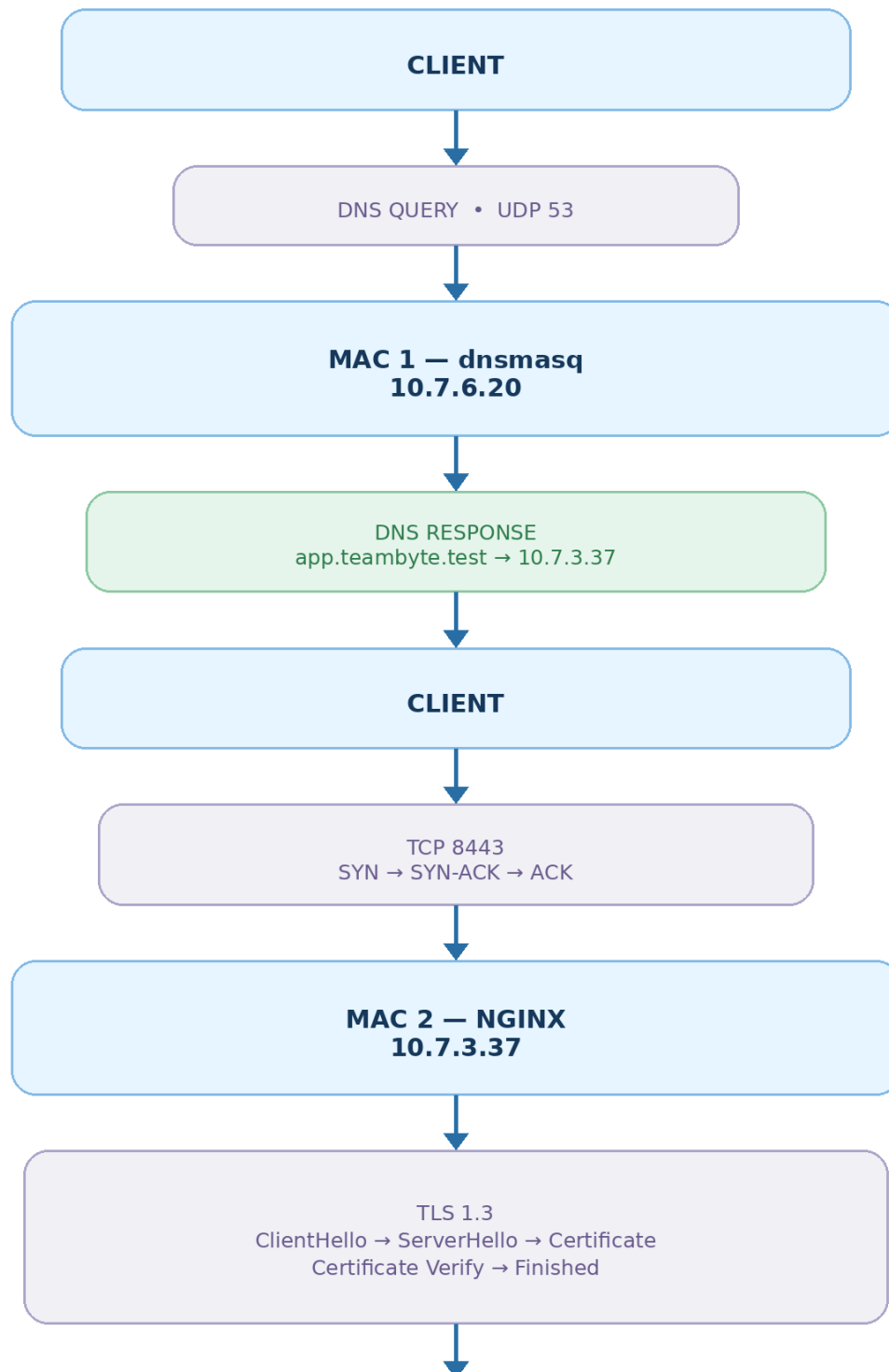
```
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % curl -i \
-H 'If-None-Match: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"' \
[http://10.7.6.20:3001/api/status
HTTP/1.1 304 Not Modified
X-Powered-By: Express
X-Backend: A
Cache-Control: max-age=60
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
Date: Sat, 03 Oct 2026 15:47:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

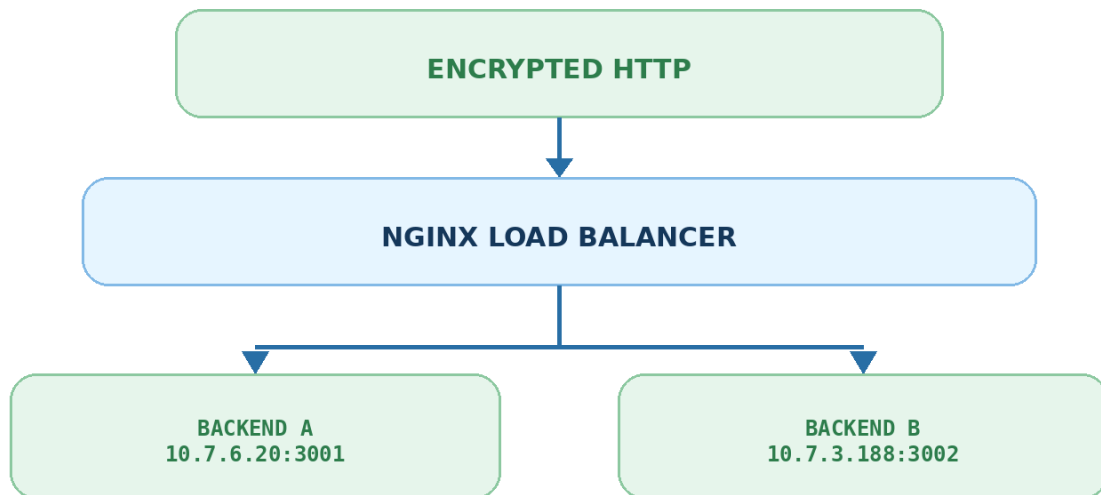
Task F Result

Task F — HTTP Caching: COMPLETE 

21. Task G — Complete Protocol Flow

The complete request flow is:





The project requires evidence of DNS, TCP handshake, TLS handshake, HTTP headers, load balancing, and port identification.

22. Wireshark — DNS Evidence



FILTER

```
dns.qry.name == "app.teambyte.test"
```

Evidence

The capture shows:

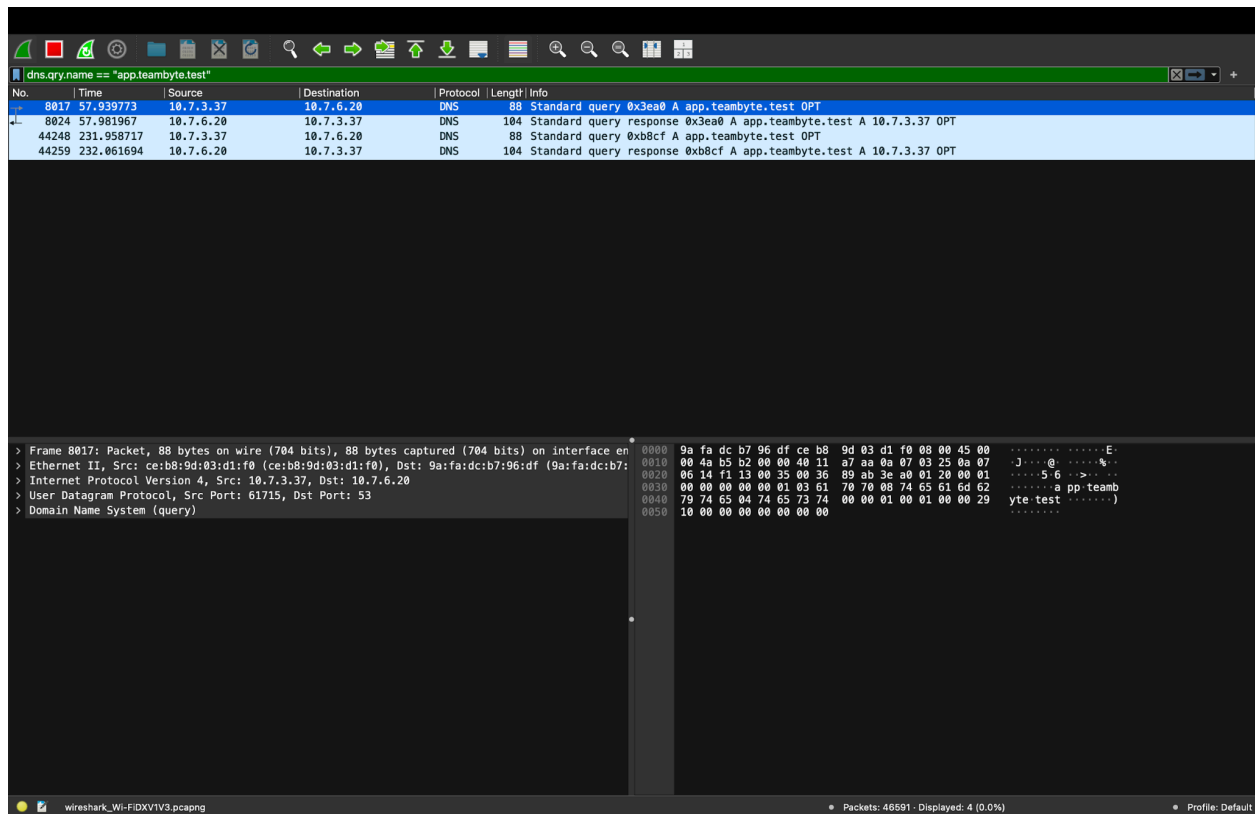
Client 10.7.3.37 → DNS Server 10.7.6.20

DNS Query: app.teambyte.test

DNS Server 10.7.6.20 → Client 10.7.3.37

DNS Response:

app.teambyte.test → 10.7.3.37



23. Wireshark — TCP and TLS Evidence



FILTER

tcp.port == 8443

The capture shows traffic between:

10.7.6.20 ↔ 10.7.3.37

TCP Three-Way Handshake

Client → Server : SYN

Server → Client : SYN-ACK

Client → Server : ACK

This establishes the TCP connection before application data is exchanged.

TLS

The capture shows TLS 1.3 handshake packets including:

ClientHello

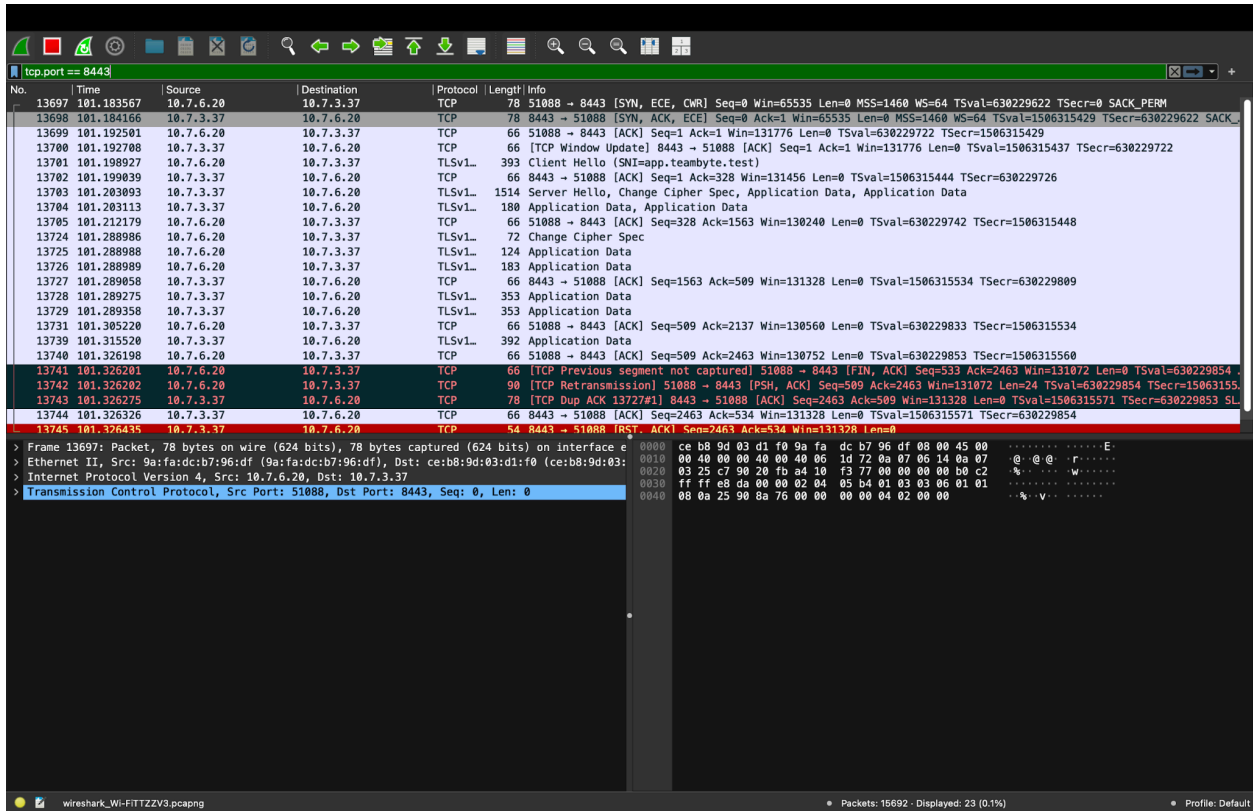
ServerHello

Certificate

Certificate Verify

Finished

After the handshake, encrypted application data is transmitted.



24. Port Identification

The important ports in the project are:

Service	Protocol	Port
DNS	UDP/TCP	53
Nginx HTTP	TCP	8080
Nginx HTTPS	TCP	8443
Backend A	TCP	3001
Backend B	TCP	3002

The client uses an **ephemeral source port**, while the server uses a known destination/service port.

For example:

Client: XXXXX → Nginx:8443

where XXXXX represents the temporary client-side port.

25. Encrypted Application Data

After the TLS handshake, HTTP application data is encrypted.

Therefore, Wireshark can show TLS/Application Data packets but cannot directly display:

```
{"backend":"A","status":"ok"}
```

as readable HTTP payload.

The encryption is provided by TLS.

26. Failure Demonstration — Backend Failure

One backend was deliberately stopped to demonstrate service continuity.

Backend B was stopped on Mac 3.

Direct access to:

10.7.3.188:3002

failed because the backend was no longer running.

However, requests through the Nginx load balancer continued to succeed through Backend A.

This demonstrates that the edge can continue serving requests through the available backend when one backend is unavailable.

```
[apoorvachoudhary@Apoorvas-MacBook-Pro ~ % curl -i http://10.7.3.188:3002/api/status
curl: (7) Failed to connect to 10.7.3.188 port 3002 after 1028 ms: Couldn't connect to server
apoorvachoudhary@Apoorvas-MacBook-Pro ~ % for i in {1..10}; do
  echo -n "Request $i: "
  curl -s -D - https://app.teambyte.test:8443/api/status -o /dev/null | grep -i "^X-Backend:"
done
Request 1: X-Backend: A
Request 2: X-Backend: A
Request 3: X-Backend: A
Request 4: X-Backend: A
Request 5: X-Backend: A
Request 6: X-Backend: A
Request 7: X-Backend: A
Request 8: X-Backend: A
Request 9: X-Backend: A
Request 10: X-Backend: A
```

27. Final Client Verification — Vidhi

After configuring Vidhi's Mac to use the private DNS server and trusting the TLS certificate:



COMMAND

```
curl -i https://app.teambyte.test:8443/api/status
```



OUTPUT

```
HTTP/1.1 200 OK
Server: nginx/1.31.6
X-Powered-By: Express
X-Backend: A
```

Cache-Control: max-age=60

ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"

{"backend":"A","status":"ok"}

This proves that Vidhi's Mac can complete the entire chain:

```
[vidhi@vidhis-MacBook-Pro ~ % curl -i https://app.teambyte.test:8443/api/status
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Sat, 03 Oct 2026 15:58:39 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 29
Connection: keep-alive
X-Powered-By: Express
X-Backend: A
Cache-Control: max-age=60
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"

{"backend":"A","status":"ok"}%
vidhi@vidhis-MacBook-Pro ~ %
```

28. Complete Phase 1 Verification

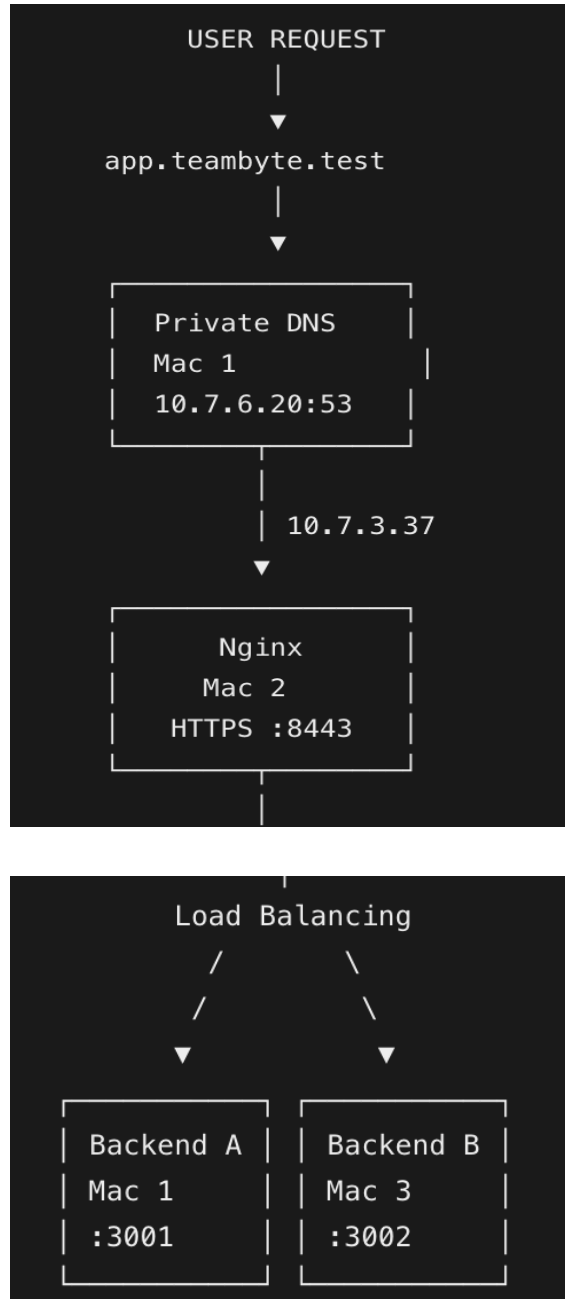
Requirement	Status
Same private LAN	✓ COMPLETE
Private IP/subnet/gateway recorded	✓ COMPLETE
Pairwise ping	✓ COMPLETE
Private DNS server	✓ COMPLETE

Requirement	Status
app.teambyte.test resolution	✓ COMPLETE
Two clients using team DNS	✓ COMPLETE
Backend A on 3001	✓ COMPLETE
Backend B on 3002	✓ COMPLETE
REST /api/status	✓ COMPLETE
X-Backend header	✓ COMPLETE
Nginx reverse proxy	✓ COMPLETE
Round-robin load balancing	✓ COMPLETE
HTTPS/TLS	✓ COMPLETE
Certificate trusted	✓ COMPLETE

Requirement	Status
No -k in final test	✓ COMPLETE
Cache-Control	✓ COMPLETE
ETag	✓ COMPLETE
304 Not Modified	✓ COMPLETE
DNS Wireshark capture	✓ COMPLETE
TCP handshake capture	✓ COMPLETE
TLS capture	✓ COMPLETE
Encrypted application data	✓ COMPLETE
Backend failure test	✓ COMPLETE

29. Phase 1 Architecture Summary

The final system works as follows:



30. Phase 1 Conclusion

Phase 1 successfully implements a fully local private network service platform.

The system demonstrates how a client request travels through multiple networking layers:

DNS → TCP → TLS → HTTP → Load Balancer → Backend

The private DNS service resolves the application domain, Nginx provides the HTTPS entry point and distributes requests between two backend servers, and HTTP caching is demonstrated using **Cache-Control**, ETag and **304 Not Modified**.

Wireshark provides packet-level evidence of DNS resolution, TCP connection establishment and TLS communication.

The project defines the Phase 1 gate as a client resolving the private domain, connecting over HTTPS, and receiving responses from both backends through the load balancer. Our implementation satisfies these requirements.
